SECOND REVIEW · ACADEMIC PROTOTYPE · NOT AN OPERATIONAL SYSTEM

Component A — Ingestion

How public Telegram posts get into the database, and how the demo still works offline.

This brief is for the teammate who presents collection. The exam path is the offline seed. Live Telegram is extra and can fail on campus Wi-Fi or Telegram rate limits.

1. What this agent does

Component A collects public Telegram posts and stores them once. It does not decide whether a post is positive, topical, or coordinated. It only records the message, the channel, the time the message was posted, and any photo that was saved.

The National Technical Research Organisation name is the origin of the public problem statement. This software is an undergraduate prototype. It is not an operational intelligence system.

2. Where it sits

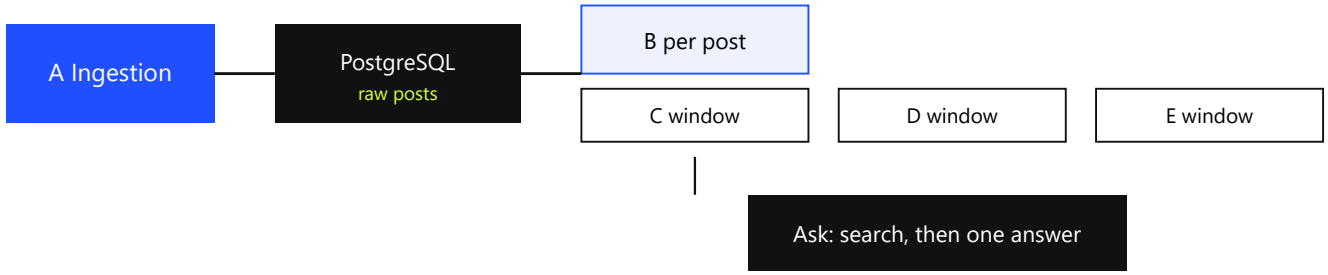


Figure 1. Posts land in Postgres. B reads one post at a time. C, D, and E recompute a time window. Ask searches the short summaries those agents write.

3. Three ways a post can arrive

Path	When to use it	What it does
Offline seed `ingestion_a/backfill.py --offline`	The viva, and any room without internet	Replays a prepared set of posts into Postgres. This is the path that must work.
Desktop export `ingestion_a/import_exported.py`	When a chat was exported from Telegram Desktop	Reads the export and inserts posts. Message id plus channel slug prevents duplicates.
Live client Pyrofork, `download_latest`	Optional demonstration only	Reads public channels, saves up to 80 recent messages, and stores photos under `Telegram Desktop`.

4. How a live pull works

1. One worker only. A second client would lock the Telegram session file.

2. Channels are read with a pause and a small random wait, so Telegram is less likely to block the account.
3. If Telegram returns a flood wait, the client sleeps for the time Telegram demands, then continues.
4. Each saved row uses `posted_at` for the message time. `ingested_at` is only when our program stored it. Later agents must use `posted_at`.
5. If the same message is imported again, the text time is kept. Photos and reaction counts can be updated.

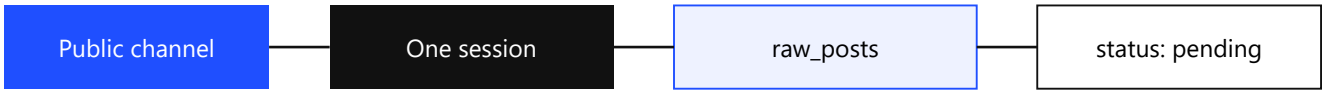


Figure 2. A new post is stored as pending. Component B is the only agent that later marks it processed.

5. What is stored

Table	Holds
`monitored_channels`	Channel name, topic tags, last check.
`raw_posts`	Text, photo path, view count, reaction counts, `posted_at`, processing status.
`raw_authors`	Public display name and bio snapshot when a discussion author is visible. No private profile scrape.

6. What to show

Open **Sources**. It lists channels, how many posts are saved, and the last check. Say: "Collection writes rows. It does not analyse them." Then show one photo path if a reviewer asks where images go: the `Telegram Desktop` folder, with the path also stored on the post.

> **One sentence.** "We store the public post and the time Telegram gave it. Analysis is a later step, so a dead network cannot erase the demo."

7. Related parts you may be asked about

Part	Relation to A
Orchestrator	Does not collect. It tells B to read pending rows, then tells C, D, and E to recompute a window.
Dashboard	Sources is view A. Overview counts saved posts and posts still waiting.
Ask	Does not read raw posts directly. It searches short summaries written after analysis.
Privacy	The design is public channels. A post-viva SQL script clears message text and display names.

8. Done, and not done

Done	Not for this review

Schema, offline replay, export import, live client with photo save and flood-wait sleep.	A full history of every channel. The live pull is capped at 80 messages and was paused.
Duplicate protection by platform message id.	No forensic hash, no deletion tracking. Both were left out of the brief on purpose.

> **Do not claim** that every chat on the phone has been downloaded, or that the live path is required for the exam.

9. Likely questions

Question	Answer
Why not the download time?	Download time measures our delay. Topic speed and coordination use `posted_at`.
Why one worker?	The Telegram session file is a single SQLite file. Two workers corrupt or lock it.
Why did a live run stop?	Telegram asked the account to wait. The client sleeps. That is rate limiting, not a crash.